Reducing Mean Time To Respond Using Large Language Model-Driven Incident Response with the Aid of Reactively Retrieved Threat Intelligence

Ayad Zewail
Faculty of Computer Science
MSA University
Giza, Egypt
ayad.ahmed@msa.edu.eg

Youssef Abdulghany
Faculty of Computer Science
MSA University
Giza, Egypt
youssef.ehab8@msa.edu.eg

Moataz Samy
Faculty of Computer Science
MSA University
Giza, Egypt
moasamy@msa.edu.eg

2025 Intelligent Methods, Systems, and Applications (IMSA) | 979-8-3315-0186-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/IMSA65733.2025.11167573

***Abstract*—Rapid response time is of paramount importance in cybersecurity incident response, yet it is currently inadequate for the growing threat landscape. Manual mitigation efforts alone are no longer sufficient t o p reserve n etwork s ecurity and neutralize emerging threats, which calls for a more automated approach to incident response. This paper proposes, implements, and evaluates a new pipeline that combines web scraping for reactive threat-mitigation instruction retrieval with a custom fine-tuned Large Language Model, specifically the Llama-3.1-8B-Lexi-Uncensored-V2 model, trained to take actions based on the retrieved instructions, which are automatically implemented by the Security Orchestration, Automation, and Response Tool. This technique promises a reduction in the Mean Time To Respond for some attacks from an average of tens of minutes to approximately 25 seconds. The study expounds upon the exact methodologies used for web scraping and Large Language Model fine-tuning and exhibits the effectiveness and efficiency o f t heir synergy.**

***Index Terms*—cybersecurity automation, large language models (LLMs), mean time to respond (MTTR), reactive threat intelligence, web scraping**

I. Introduction

In the context of incident response, the most valuable asset that could be the decider between mitigation and a breach is time. Taking too long to notice, identify, and respond to an incident can cause major losses, and is unfortunately very common as 82% of enterprises view their security tools as insufficient f or r esponding t o i ncidents i n a t imely manner [1]. Moreover, the rate at which cyber threats are growing and improving is causing the average Mean Time To Respond (MTTR) to increase, which puts enterprises at even higher risk of being breached and compromised [2]. Furthermore, the matter of delayed mitigation and increased vulnerability becomes worse when manual analyst work is taken into account, as manual response alone is liable to natural human errors and is also very time consuming [3]. These consequences call for the improvement and implementation of a more efficient and robust system that can automate the mitigation of threats that are both generic and severe. This paper will argue for an innovative framework that utilizes web scraping technologies with Large Language Model (LLM) fine-tuning methods that were built on top of a custom-made Security Orchestration, Automation, and Response (SOAR) tool, to achieve a comprehensive solution that has the potential to not only mitigate generic cyber threats but also more severe or unique threats as well. This paper will discuss the methodologies and technologies used in the creation of this framework, as well as provide insightful statistics that highlight the capabilities of this framework and how it can be used to reduce MTTR and automate incident response.

979-8-3315-0186-0/25/31.00©2025IEEE

II. Literature Review

A. Reducing MTTR in Incident Response Using AI

MTTR, which is a measurement of the time taken to mitigate a cyber attack on a system or infrastructure, is considered to be of utmost importance when dealing with a potential breach, as it directly corresponds to the odds of the attack being successful, and the more it is reduced, the less the chances of the attacker being able to breach the system. While traditional methods relying on manual threat detection and human mitigation expertise have achieved varying degrees of success, it appears they reached their limits when it comes to the improvements that could be made. However, a promising new trend is the utilization of AI technology that is now being used to reduce MTTR. Among the attempts that focus on improving MTTR using AI indicate how traditional MTTR methods are becoming insufficient due to the various difficulties that are caused by a shortage of experts and alert fatigue. The aim is to introduce AI-driven solutions such as Isolation Forest model that isolates outliers in high-dimensional data such as network traffic logs, Autoencoders to detect anomalies by reformatting regular behavioral patterns, and K-Mean clustering to essentially segregate the data into groups allowing for ease of detection for outliers in login times. Other uses of AI technology were the use of a BERT model or LLM to parse threat reports. Overall, they were able to reduce MTTR from 4.6 hours to 2.4 hours for highly complex attacks [4]. Additionally, there is potential in the use

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:35 UTC from IEEE Xplore. Restrictions apply.

of unsupervised machine learning methods such as neural networks and clustering to detect incidents and help reduce false positives as well as detecting zero-day attacks by analyzing network traffic and user behavior patterns. Subsequently, an overall reduction in MTTR compared to traditional rule-based approaches [5]. However, the main limitation of these attempts is that they still rely on humans to mitigate the attack which is prone to error and can be extremely slow, and with the threat landscape growing ever more, it becomes harder for cybersecurity analysts to keep up and be competent to handle the threats.

B. Web Scraping for Intelligence Retrieval

Web scraping is a powerful tool used to extract all kinds of data from websites in an automated manner. The utilization of web scraping in the context of cybersecurity for the procurement of threat intelligence is a very common and reliable method; this is due to its ability to automate open-source intelligence (OSINT). Yet, most of the implementations, if not all, of web scraping in intelligence are used in a proactive way rather than a reactive way; Threat intelligence is seen as an asset used prior to attacks to aid in strategic decision-making [6]. Similarly, other implementations focus heavily on getting intelligence upfront to gain the ability to later deal with attacks should they occur [7], [8]. Some implementations include using Hypertext Transfer Protocol (HTTP) requests alongside Beautiful Soup for parsing to form a simple yet structured and effective procedure to scrape intelligence [6] while others deploy more complex and extensible methods to gather more detailed and informative intelligence regarding the threat, such as "Tweepy" for social media data, "Scrapy" for web crawling, "Terraform" for infrastructure management, and many others [7], [8]. However, some works put forward implementations that could be used for reactive use, but no explicit attempts or considerations were mentioned [9]. The reactive potential of web scraping in incident response is overlooked despite it granting the possibility to tailor the acquired intelligence to the undergoing attack instead of relying on pre-acquired intelligence that may possibly not match well with the attack or may lack sufficient information about the actual attack. This paper will address this gap by implementing web scraping through reactive search queries that allows for full dynamism.

C. The Use of LLMs in Incident Response

Large Language Models (LLMs) are advanced Artificial Intelligence (AI) algorithms trained on relatively large amounts of data such as text, to understand and generate human-like language; Examples include ChatGPT, Gemini, Llama, and Claude. First, regarding existing literature, the integration or "employment" of LLMs for updating Incident Response plans (IRPs) and standard operating procedures (SOP) has been introduced. These included LLM-automated drafting, a way to utilize ChatGPT to create initial IRP/SOP drafts, Gap Identification, detection of missing SOPs, Visual Workflows and Post-incident Improvement where analysis of incident logs shows ways to improve in order to reduce the occurrence of these incidents [10]. Moreover, proposals for LLM-based multi-agent systems using ChatGpt-4o to essentially act as a collaboration between experts and LLM models for incident response have been put forward, and it highlights how LLMs can play more traditional human roles in cybersecurity situations such as endpoint analysis and how LLMs can work alongside humans to assist them with the more repetitive tasks of incident response scenarios [11]. However, the solutions offered give LLMs more background-level tasks and make them recommend solutions instead of utilizing them to directly enforce mitigation actions to remediate attacks. This paper proposes giving the LLM complete authority to implement actions based on what it sees fit.

III. Methodology

This paper proposes a framework that performs reactive web scraping to dynamically obtain mitigation instructions that are then passed to a custom fine-tuned LLM to produce an output in JSON format and then executed by a custom SOAR tool to neutralize the threat, as illustrated in Fig. 1. The goal is to develop a robust and adaptable system that can handle a wide range of known attack vectors. This section will delve into the details of the methods used, their implementations, and their role in this pipeline.

A. Attack Detection Using Scapy and Snort

The first and most important step in mitigating an attack is its detection in the first place. This pipeline deploys Scapy for monitoring and Snort for threat detection. Firstly, Scapy is a Python library that allows for packet sniffing and analysis for each packet entering the network; this allows for vigilant monitoring of the network to detect any attacks that may occur. It also allows for packet fabrication, which helps in simulating attacks on the network. Secondly, Snort, a Cisco-maintained Intrusion Detection System (IDS) that utilizes rule-based pattern matching to detect known attacks in real-time. Snort is an excellent and reliable IDS for this pipeline, as it is capable of effectively detecting a vast amount of attack

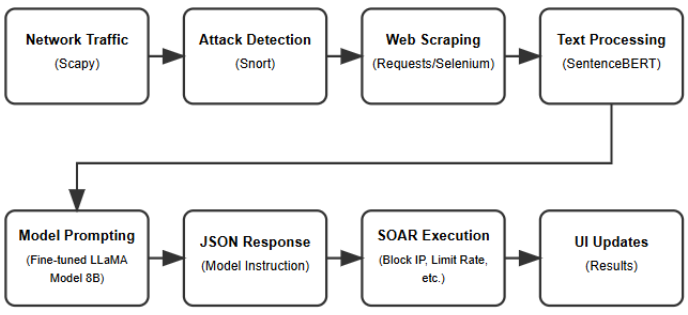
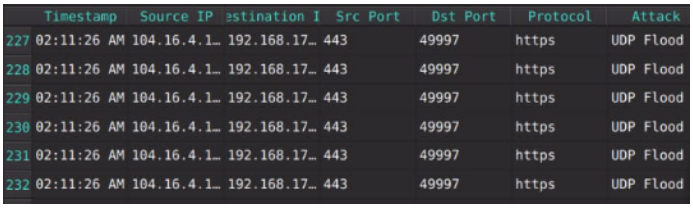


Fig. 1. Visual Depiction of the Proposed Pipeline.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:35 UTC from IEEE Xplore. Restrictions apply.



	Timestamp	Source IP	estination I	Src Port	Dst Port	Protocol	Attack
227	02:11:26 AM	104.16.4.1…	192.168.17…	443	49997	https	UDP Flood
228	02:11:26 AM	104.16.4.1…	192.168.17…	443	49997	https	UDP Flood
229	02:11:26 AM	104.16.4.1…	192.168.17…	443	49997	https	UDP Flood
230	02:11:26 AM	104.16.4.1…	192.168.17…	443	49997	https	UDP Flood
231	02:11:26 AM	104.16.4.1…	192.168.17…	443	49997	https	UDP Flood
232	02:11:26 AM	104.16.4.1…	192.168.17…	443	49997	https	UDP Flood

Fig. 2. Display of the detected attack on the network.

types automatically; when this is paired with Scapy, it allows for very quick and perceptive monitoring capabilities to begin acting towards mitigation as soon as possible, which serves MTTR quite well over manual threat detection. After an attack is detected, it is placed in a special table and displays its information during the mitigation process as in Fig. 2

Once the attack is logged, it is ready for the next stage where attack intelligence is retrieved through web scraping.

B. Dynamic Retrieval of Threat Intelligence with Web Scraping and Document Processing

In order to create a system that can dynamically and independently adapt to deal with any threat it faces, it needs to have the capabilities to acquire data through web scraping and process those data. To achieve this, a web scraping script is developed that receives the attack name from the alert to construct the query with which it will perform its search; the query is in the form of "ATTACK_NAME Mitigation and Response". Afterwards, it commences the search which is performed through the bing search engine, chosen specifically in an attempt to avoid google's rigorous anti-scraping defenses, using asynchronous requests and beautiful soup as the main method for speed and efficiency since it can retrieve and parse multiple pages at once, but in the case the search results are loaded through JavaScript, a backup Selenium function is ready to be called to launch a highly optimized headless search and retrieve all of the result links of the search. Subsequently, each link is parsed to retrieve the paragraphs inside of that website and is stored in a CSV file with the format: (link, text). It is worth noting that this particular method of web scraping through a direct search is used since it proved to be the most effective and reliable; Alternative methods, such as deploying dedicated spiders in specific websites, were found to yield subpar results as this pipeline requires direct and actionable sentences that give a sense of instruction. However, the presence of such sentences on defense-oriented websites such as 'defend.mitre.org', 'OWASP', 'CISA', etc. was limited to some attacks such as 'Denial of Service' without others, unhelpful such as CISA's 'Apply mitigations per vendor instructions or discontinue use of the product if mitigations are unavailable', or presented in ways impossible for the LLM to interpret, such as charts and graphs. After web scraping, there remains the CSV file to process in order to extract the true mitigation sentences from among the scraped paragraphs. This is performed with spaCy's sentence tokenizer which splits paragraphs into sentences in a more efficient way than other

```
Block a remote malicious IP by blocking outgoing
communications.
Block an external IP address that is attacking the
corporate network by blocking incoming communications.
Block traffic to and from IP addresses identified as
malicious by trusted sources.
Create a network security rule to block outgoing
traffic to malicious IP addresses.
Network protection can block access to malicious IPs or
domains.
Update the Network Firewall rule group to block traffic
to malicious IP addresses.
Use a network ACL to block specific IP addresses from
accessing.
```

Fig. 3. Sample of the mitigation reference list.

alternatives such as NLTK. With the sentences ready, it is possible to find which provide mitigation instructions. In this stage, the SentenceTransformer Python library is used to load the SentenceBERT model, specifically the "all-MiniLM-L6-v2" model, which uses cosine similarity to compare sentences with a custom-made reference list that includes numerous sentences that resemble instructions, as shown in Fig. 3.

Initially, the reference list was hand-crafted and small in size which proved to be insufficient and returned many false positives. The new list was constructed from real sentences gathered from blogs, social media posts, organizational websites, etc. to ensure that the real structure of the instructions were captured. The cosine similarity is able to identify sentences that look semantically similar to the desired mitigation style even if there are some differences in the structure or wording thanks to SentenceBERT's capabilities. After scoring the sentences, the top-scoring sentence becomes the prompt that will be sent to the LLM.

C. LLM Fine-Tuning

Once the web scraping is complete and the information is retrieved, it is sent to a Llama-3.1-8B-Lexi-Uncensored-V2 model [12] that is fine-tuned to return output in JSON format which will be executed by a custom SOAR tool. First, a brief overview of the model itself will be provided. Then, details regarding the process of fine-tuning, including the process of quantization and Low-Rank Adaptation (LoRA) fine-tuning, will be discussed.

1) Model Overview:

- **Base Architecture:** Derived from Meta's Llama-3.1-8B-Instruct.
- **Parameters:** 8.03 billion.
- **Tensor Type:** BF16 (bfloat16) for efficient computation.
- **License:** Governed by the Meta Llama 3.1 Community License that permits commercial use with specific attribution and compliance requirements.
- **Uncensored Nature:** Fine-tuned to minimize refusals and enhance compliance out of the box.

2) Performance Metrics:

It is important to note that the characteristics of the model shown in Table I are those of the original model

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:35 UTC from IEEE Xplore. Restrictions apply.

TABLE I
BENCHMARK SCORES (OPEN LLM LEADERBOARD)

Metric	Results
IFEval	77.92% (strict accuracy)
BBH (3-Shot)	29.69% (normalized acc.)
MATH Lvl 5 (4-Shot)	16.92% (exact match)
MMLU-PRO (5-Shot)	30.90% (accuracy)
GPQA (0-Shot)	4.36% (acc_norm)
MuSR (0-Shot)	7.77% (acc_norm)

before applying the necessary fine-tuning required for the proposed pipeline.

3) An Overview of Quantization and LoRA:

Quantization is the process of normalizing the size of the number of bits used to represent the model's weights; this results in the model being easier to load and manage. However, this is at the cost of the model's accuracy of response.

Represented by: $q = \text{round}\left(\frac{x}{s}\right) + z$, Low-Rank Adaptation (LoRA) is a popular method to fine-tuning LLMs.

Traditional fine-tuning works by loading the model and training it by providing the model with a series of user prompts and the expected output for said prompts directly below them; LoRA instead focuses primarily on maintaining a record (adaptor) of the changes for the model's weights. This record is primarily stored in a matrix much like the model's actual weights; the record is then decomposed into two matrices through a process known as matrix decomposition which is represented by $\Delta W \approx AB$, where:

$$A \in \mathbb{R}^{d\times r}, \quad B \in \mathbb{R}^{r\times k}, \quad \text{and} \quad r \ll \min(d,k). \quad (1)$$

The updated weight is given by:

$$W' = W + AB. \quad (2)$$

In other words, instead of maintaining a full record of the changes to the weights, LoRA opts for two smaller matrices that are then cross multiplied into the original record; this makes the process of fine-tuning significantly more efficient by a factor of $log(d)$. This process can also be combined with quantization allowing for further optimization in resource usage when it comes to fine-tuning; this method is called Quantized Low-Rank Adaptation (QLoRA), where the values of the decomposed matrices are normalized from 32 bits to 8 or 4 bits depending on the configuration. Additionally, the model itself is quantized during training to ensure that it is being tested and evaluated in its quantized form; this is known as Training-Aware Quantization (TAQ). After the training is completed, the next step is to merge the adapter with the model, which essentially modifies its weights. In the case of this paper, the model is loaded in its quantized form, then the adapter is merged: This is done due to the model's high demand for VRAMS. Quantizing the model before merging it with the adapter enables the management of the model's high VRAM demand on the available hardware provided by Kaggle notebooks.

4) QLoRA Fine-Tuning Configuration Summary:

- **LoRA Hyperparameters**
  - Rank (`lora_r`): 64
  - Alpha (`lora_alpha`): 16
  - Dropout (`lora_dropout`): 0.1
- **Quantization**
  - 4-bit Precision: Enabled (`use_4bit` = `True`)
    - Quant Type: `nf4` (NormalFloat4)
    - Compute Dtype
    - Nested Quantization: Disabled (`use_nested_quant` = `False`)
- **Training Arguments**
  - Batch Sizes: 4 (train/eval) + `gradient_accumulation_steps` = 1 → Effective batch size = 4
  - Optimizer: `paged_adamw_32bit`
  - Learning Rate: 2×10^{-4} with cosine scheduling + `warmup_ratio` = 0.03
  - Regularization:
    - `weight_decay` = 0.001
    - `max_grad_norm` = 0.3
  - Efficiency:
    - `gradient_checkpointing` = `True`
    - `group_by_length` = `True`
- **SFT (Supervised Fine-Tuning)**
  - Sequence Length: `max_seq_length` = `None`
  - Packing: Disabled (`packing` = `False`)

5) Training Data and Expected Outputs:

The Llama model has a specific chat template that is used to fine-tune it; the "user" is the prompt, and the "assistant" is the model's response. The model was trained on the same reference list of mitigation instructions that the web scraping uses[1], followed by the expected JSON output for each query, as shown in Listing 1.

```
<|start_header_id|>user<|end_header_id|>
Implementing rate limiting to restrict the
    number of requests from specific IP
    addresses or sources within a given
    timeframe can be effective. 172.16.254.1
    34567
<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
{"function": "set_rate_limit", "parameter": "10"
    , "parameter2": "172.16.254.1"}
<|eot_id|>
```

Listing 1. Example of the Llama chat template used for fine-tuning

[1]This ensures that the web scraping retrieves sentences semantically resembling the high-quality data the model was trained on in content and structure.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:35 UTC from IEEE Xplore. Restrictions apply.

TABLE II
EVALUATION METRICS AND THEIR INTERPRETATION

Metric	Value	Interpretation
Exact Match	76.7%	Outputs matched gold standards in 23/30 cases
Schema Compliance	96.7%	29/30 responses followed JSON schema requirements
Field-wise F_1	0.92	High precision/recall for individual field parsing

The JSON output is then parsed and dynamically executed without having to set additional conditions in the code to execute it as in Listing 2.

```
def extract_function_and_params(self,
    model_output):
   data = json.loads(json_text)
   values = list(data.values()) if isinstance(
       data, dict) else None
   self.execute_function(self.MitEng, values[0],
        *values[1:])

def execute_function(self, obj, function_name, *
    args, **kwargs):
   func = getattr(obj, function_name, None)
   return func(*args, **kwargs)
```

Listing 2. Python function to extract and execute

This allows for a dynamic way to receive functions from the LLM which is typically run on a Kaggle notebook due to the demanding resource requirements. The functions consist of common SOAR methods to mitigate attacks such as: blocking IPs, blocking ports, rate limiting, terminating processes, etc. With the help of the aforementioned step, the model is able to directly interact with the SOAR tool without the user's explicit consent, providing a seamless and intuitive 'autopilot' functionality that can be toggled by the user.

6) Evaluation Results:
Given that the model is fine-tuned to produce JSON outputs with no further text, a more innovative way of measuring the model is in order; consequently, the following measurements were developed as in Table II:

- **Exact Match**: Measures how often the model produces the expected JSON output.
- **Schema Compliance**: Measures how well the model followed the actual structure. For example, most JSON outputs are expected to have a definition called `"function"` and a `"parameter"` field, which describes the parameter(s) of the function. In some cases, multiple parameters are expected if the function requires several parameters.

IV. EXPERIMENTS AND RESULTS

With the pipeline implemented and ready for testing, the evaluation will be performed by putting the pipeline against several attacks and comparing the MTTR of the tool with the average of the manual methods that are inferred from industry norms, anecdotal accounts, and publicly shared experiences. For this paper, 3 popular attacks will be used for testing.

A. Denial of Service Attack

Firstly, a Denial of Service (DoS) attack. Manual mitigation of DoS attacks takes an average of approximately 30 minutes, depending on the given circumstances. When the pipeline was exposed to it, it performed web scraping and returned the following as the top sentence:

> "Rate limiting refers to restrictions on the number of requests from a specific IP address within a timeframe."

This, when passed to the LLM, yielded the function "limit_rate" and the pipeline automatically executed the function, achieving a total runtime of 27.15 seconds as shown in the autopilot log in Fig. 4.

B. SSH Brute-Force Attack

Secondly, a brute-force attack was performed through SSH. This attack takes around 15 to 30 minutes to mitigate manually on average. When detected and the pipeline initiated, the following sentence was yielded as the result:

> "IP blacklisting can block known malicious IP addresses that have been associated with brute force attacks in the past."

The LLM then received this prompt and returned "block_ip" after a total of 26.17 seconds as shown in Fig. 5.

C. Port Scanning Attack

Thirdly, a port scanning attack was performed. Manual detection and mitigation of port scanning typically take between 30 minutes to 2 hours. When the pipeline was subjected to it, it scraped the following sentence:

> "When properly configured, a firewall can effectively block unauthorized access attempts, filter network traffic, and provide an extra layer of defense against port scanning attacks."

When prompted, the LLM returned "block_ip", achieving a total runtime of 27.68 seconds as shown in Fig. 6.

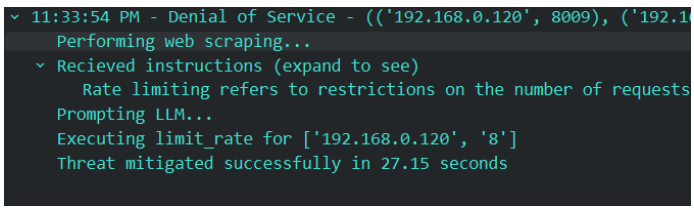


Fig. 4. Denial of Service Mitigation Pipeline.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:35 UTC from IEEE Xplore. Restrictions apply.

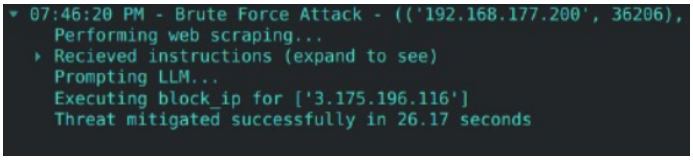


Fig. 5. SSH Brute Force Mitigation Pipeline.

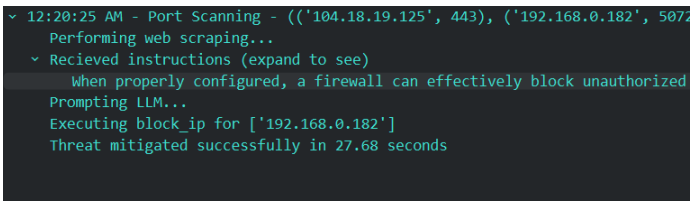


Fig. 6. Port Scanning Mitigation Pipeline.

V. Discussion

The significant reduction in MTTR with this pipeline proves that the use of AI-driven response for speed and reactive intelligence gathering for flexibility are more effective than manual methods and worth further investment in research. Furthermore, the results show the feasibility of the pipeline and promise a real impact in the field of cybersecurity incident response, as MTTR of some attacks was reduced from an average of half an hour to half a minute, an approximate 98% reduction that can help tremendously eliminate attackers and protect the network in sufficient time. Other works that attempted to reduce MTTR through automation have achieved a high MTTR reduction such as PHOENI2X at 96 - 98% [13], but this pipeline surpasses it in adaptability as instructions are dynamically acquired based on the incident, not statically made with predefined playbooks. Initially, the LLM was overfitted due to the small hand-crafted training data, but after improving both the quantity and quality of the data to use realistic sentences and the usage of the training data as the reference list for the web scraping, results were significantly improved, as the pipeline retrieved sentences semantically similar to the training data and passed them onto a model that was trained on higher quality data.

VI. Conclusion

In conclusion, the manual approach to cyber threat mitigation is insufficient for the dangerous landscape enterprises are exposed to, and thus necessitates the introduction of a new pipeline that emphasizes efficiency and robustness through the synergy of reactive web scraping and LLM-driven decision making. Related work in the field integrated AI in the mitigation process to automate detection and parsing. In addition, web scraping was utilized for OSINT, but it was focused on proactive intelligence gathering. Regarding LLMs, they were used for proactive planning more than for reactive decision-making. This pipeline tackles these points by performing reactive procedures that scrapes the needed intelligence from the web and utilizes a decision-making LLM fine-tuned with QLoRA to successfully reduce the MTTR of some of the common attacks from tens of minutes down to $\sim$25 seconds and provide an adaptable and efficient mitigation method.

VII. Future Work

Despite this paper's step forward towards reducing MTTR through automated reactive methods, there is more that can be done to improve the capabilities of the pipeline. Future development may benefit from implementing ML-based anomaly detection that is equipped with reinforcement learning to be able to handle zero-day attacks to be even more robust and reliable. Additionally, more complex web scraping can be implemented to utilize more information extracted from the malicious packet such as the destination port, the decoded payload, the headers, etc. to provide a more specialized search for specific attack circumstances. Further enhancements to the fine-tuning are also in order, such as adjusting quantization parameters or rank settings, considering different emerging fine-tuning methods such as P-tuning v2, using larger and more curated datasets to avoid overfitting, and implementing a continuous fine-tuning pipeline such that the model can be trained as soon as new attack types emerge.

References

[1] Cyber Defense Magazine, "Uncovering the gaps in cyberthreat detection and the hidden weaknesses of SIEM," *Cyber Defense Magazine*, 2024. [Online]. Available: https://www.cyberdefensemagazine.com/uncovering-the-gaps-in-cyberthreat-detection-the-hidden-weaknesses-of-siem/

[2] A. Johnson, "Improving incident response times through efficient security operations center (SOC) management techniques," *Int. J. Core Eng. Manag.*, vol. 7, no. 1, pp. 1–10, 2016.

[3] C. Onwubiko and K. Ouazzane, "Challenges towards building an effective cyber security operations centre," *arXiv*, 2022. [Online]. Available: https://arxiv.org/abs/2202.03691

[4] Z. Yousaf and D. Boomsma, "AI-Driven SOC Operations: Improving Incident Response Time and Threat Analysis," 2024. [Online]. Available: https://www.researchgate.net/publication/389336472

[5] I. Khalid and M. S. Purdie, "AI-powered SOC operations: revolutionizing cyber security incident response and management," *Res. Gate*, 2024, doi:10.13140/RG.2.2.33339.53287.

[6] A. A. Nugraha and D. Kristomo, "Python web scraping for threat intelligence," *KESATRIA: J. Penerapan Sist. Inf.*, vol. 5, no. 2, pp. 714–719, 2024. [Online]. Available: https://www.researchgate.net/publication/381316575_Python_Web_Scraping_for_Threat_Intelligence

[7] P. Balasubramanian, S. Venkatakrishnan, and R. Gupta, "TSTEM: a cognitive platform for collecting cyber threat intelligence in the wild," *arXiv*, 2024. [Online]. Available: https://arxiv.org/abs/2402.09973

[8] P. Kuehn, D. Nadermahmoodi, M. Bayer, and C. Reuter, "ThreatCrawl: A BERT-based Focused Crawler for the Cybersecurity Domain," arXiv preprint arXiv:2304.11960, revised Jan. 17, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2304.11960

[9] K. Satvat, R. Gjomemo, and V. Venkatakrishnan, "Extractor: extracting attack behavior from threat reports," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*, 2021, pp. 598–615. doi: 10.1109/EUROSP51992.2021.00046

[10] S. Hays and J. White, "Employing LLMs for incident response planning and review," *arXiv*, 2024, doi:10.48550/arXiv.2403.01271.

[11] Z. Liu, "Multi-agent collaboration in incident response with large language models," in *AAAI Workshop on MARW*, 2025. [Online]. Available: https://arxiv.org/abs/2412.00652

[12] Orenguteng, "Llama-3.1-8B-Lexi-Uncensored-V2," Hugging Face. [Online]. Available: https://huggingface.co/Orenguteng/Llama-3.1-8B-Lexi-Uncensored-V2

[13] A. Lekidis, V. Mavroeidis, and K. Fysarakis, "Towards incident response orchestration and automation for the Advanced Metering Infrastructure," arXiv, 2024. [Online]. Available: https://arxiv.org/abs/2403.06907

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:38:35 UTC from IEEE Xplore. Restrictions apply.